

Push Button and Signal Modulation in Raspberry-pi

Agenda

- Push button
- Buzzer
- Signal Modulation
- Pulse Width Modulation (PWM)

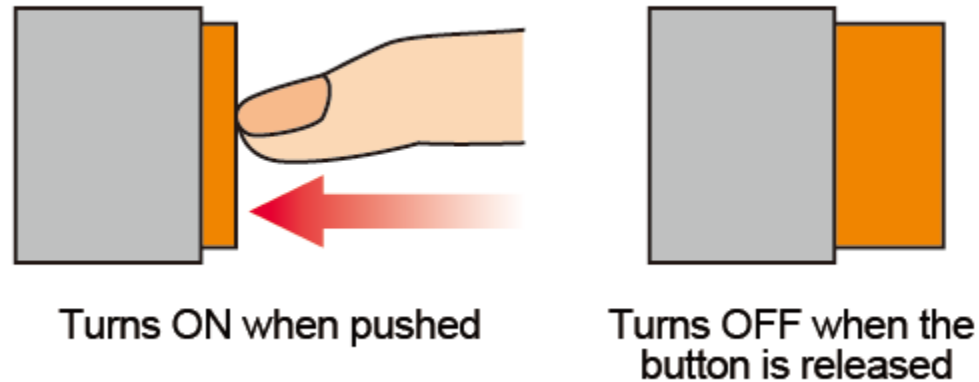
Push Button

- Push buttons are switch devices that are manually operated.
- A Push Button switch is a type of switch which consists of a simple electric mechanism to turn something on or off.
- Depending on model they could operate with momentary or latching action function.
- Calculator buttons, Reset switches, Stopping machinery



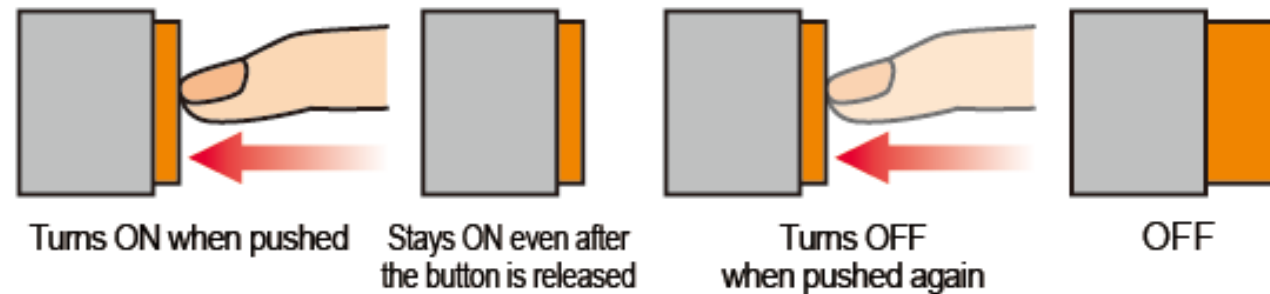
Push Button Operations: Momentary

- The button returns to its original position after it is released.
- the momentary operation method keeps the ON state of a switch only when its button is pressed and held, and the switch returns to its OFF state when the button is released, which is also referred to as a self-reset type.



Push Button Operations: Alternate

- A switch of this type turns ON when its button is pressed once, and maintains its ON state even after the button is released. This operation method is referred to as the self holding type



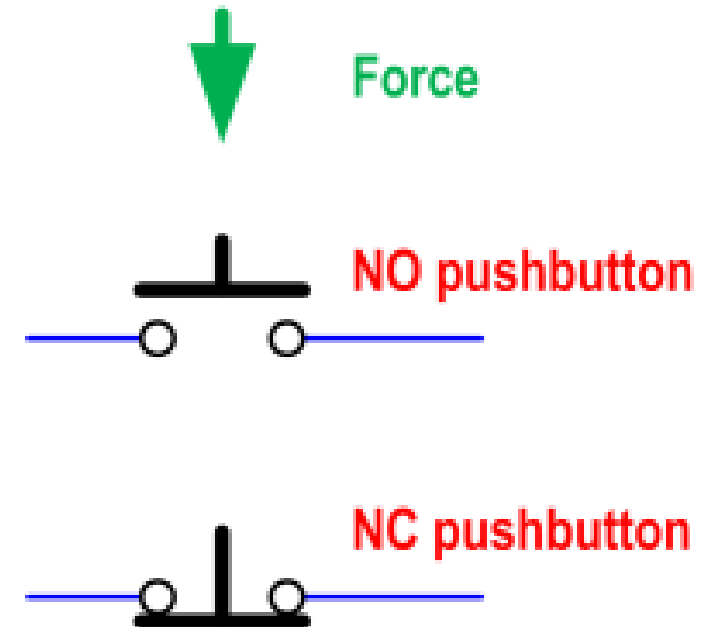
Push Button Operations: Push-Pull

- When the button is pressed, an internal lock mechanism holds it in the same position. The button is returned to its original position when it is pulled to released the lock.



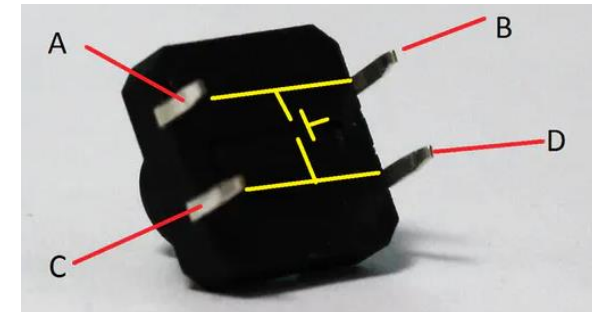
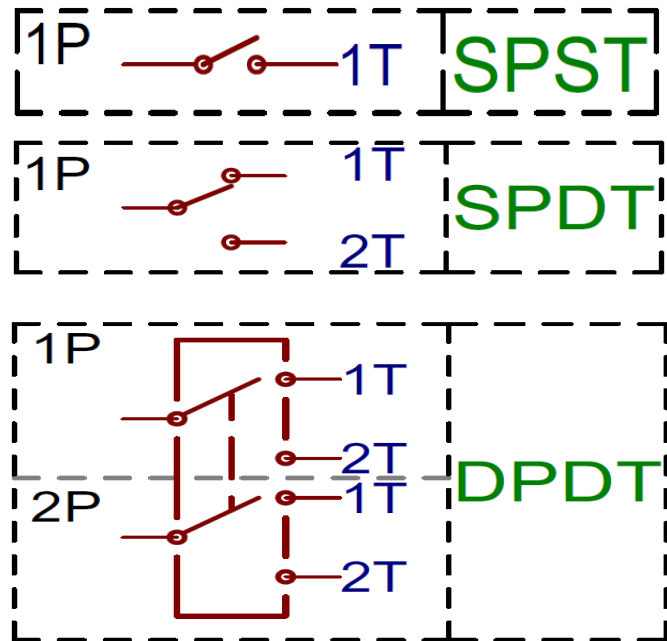
Push Button Types

- Normally Open: When an NO button is pressed, you provide a path for the current.
- Normally closed: When an NC button is pressed, you impede the current from flowing.

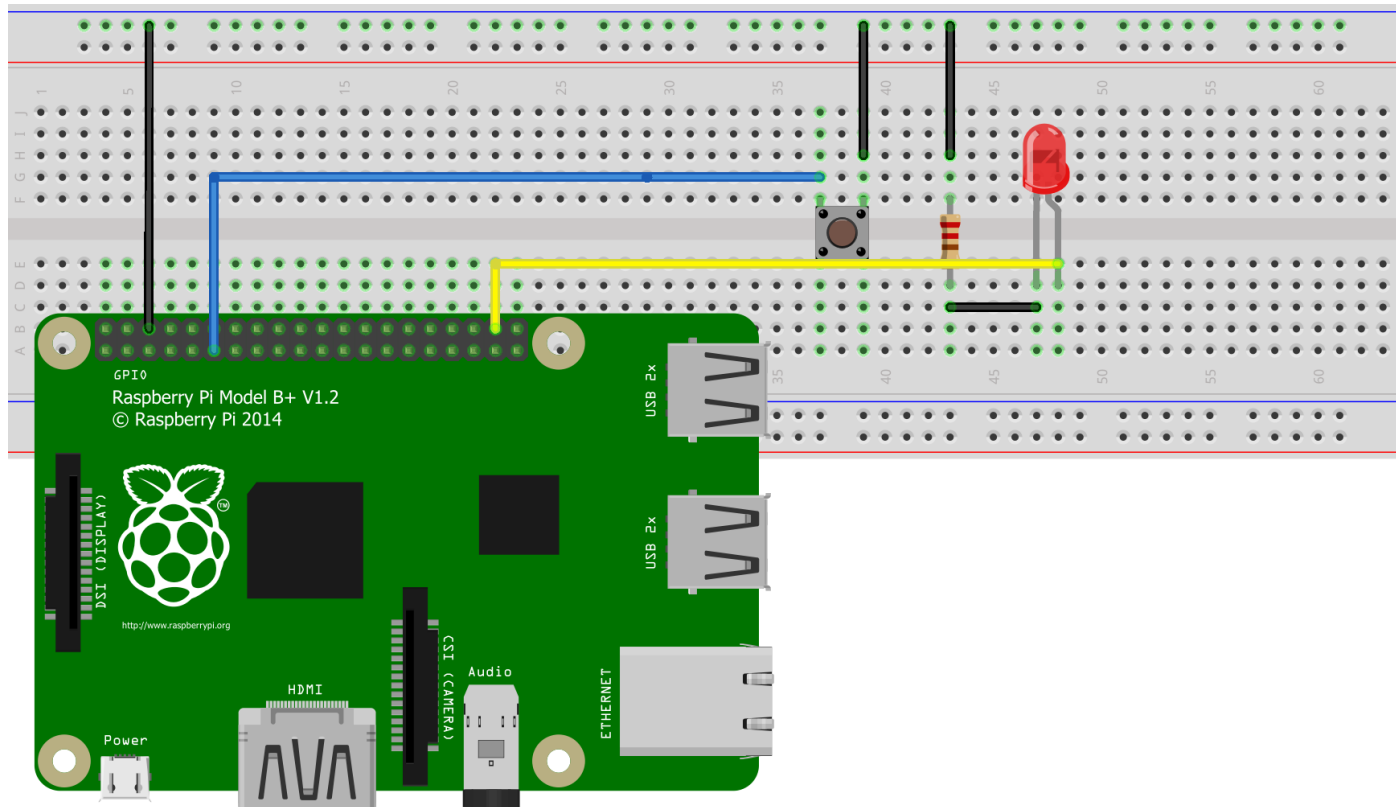


Button Input/Output

- Pole refers to the number of input circuits of a switch
- Throw refers to the number of output circuit of a switch



Push Button Circuit



fritzing

Push Button Code

```
1.  import RPi.GPIO as GPIO
2.  import time
3.  GPIO.setmode(GPIO.BCM)
4.  GPIO.setup(24, GPIO.OUT)                #LED to GPIO24
5.  GPIO.setup(23, GPIO.IN, pull_up_down=GPIO.PUD_UP)#Button to GPIO23
6.  while True:
7.      button_state = GPIO.input(23)
8.      if button_state == False:
9.          GPIO.output(24, True)
10.         print('Button Pressed...')
11.         time.sleep(0.2)
12.     else:
13.         GPIO.output(24, False)
```

Buzzer

It is essentially a sound-emitting device traditionally used as a signal device.

A passive buzzer is an electromagnetic squeaker used to generate sound signals of different frequencies. It needs a signal source that provides the sound signal

An active buzzer generates the sound itself (internal oscillator). It is the simplest module to produce a sound of about 2 kHz.

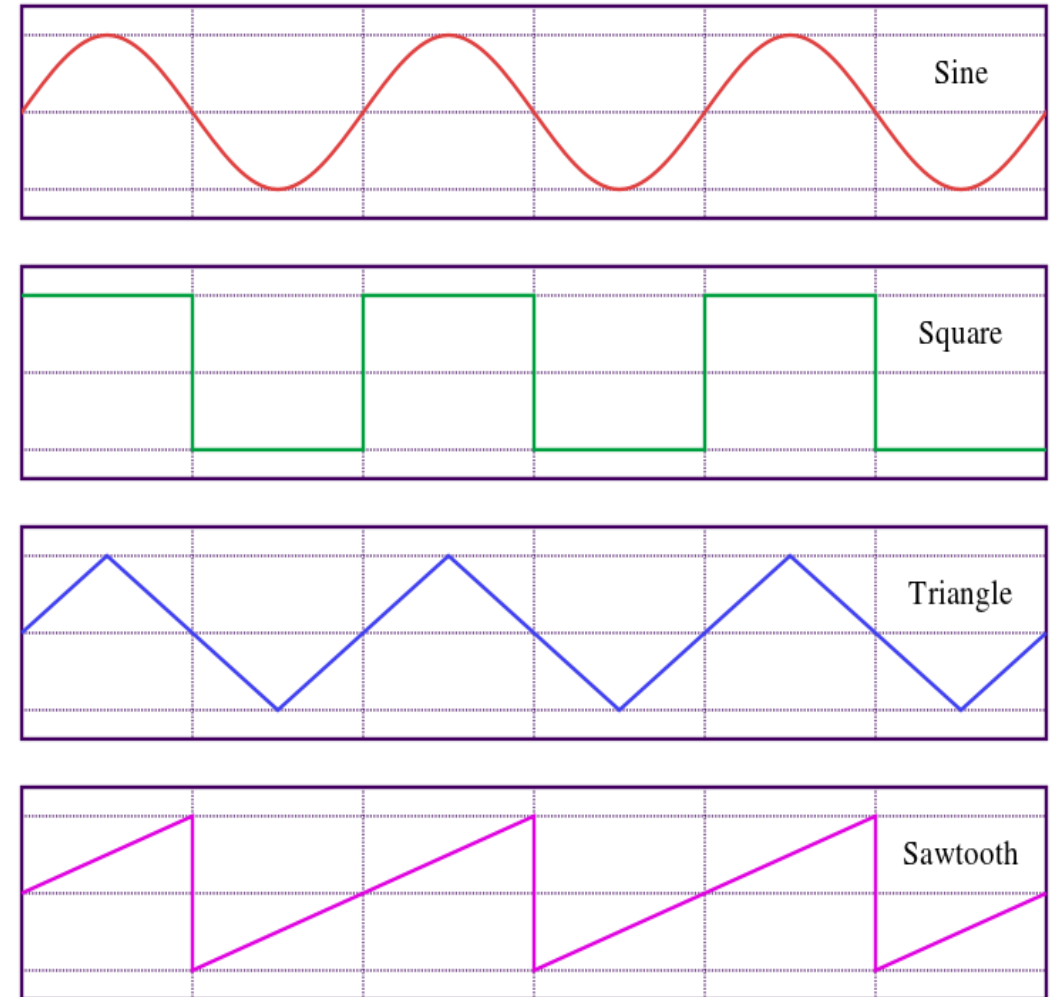


Buzzer Python Code

```
import RPi.GPIO as GPIO  
import time  
Buzzer = 21  
GPIO.setmode(GPIO.BCM)  
GPIO.setup(BuzzerPin, GPIO.OUT)  
GPIO.setwarnings(False)  
while 1:  
    GPIO.output(buzzer,GPIO.HIGH)  
    sleep(1)  
    GPIO.output(buzzer,GPIO.LOW)  
    sleep(0.5)
```

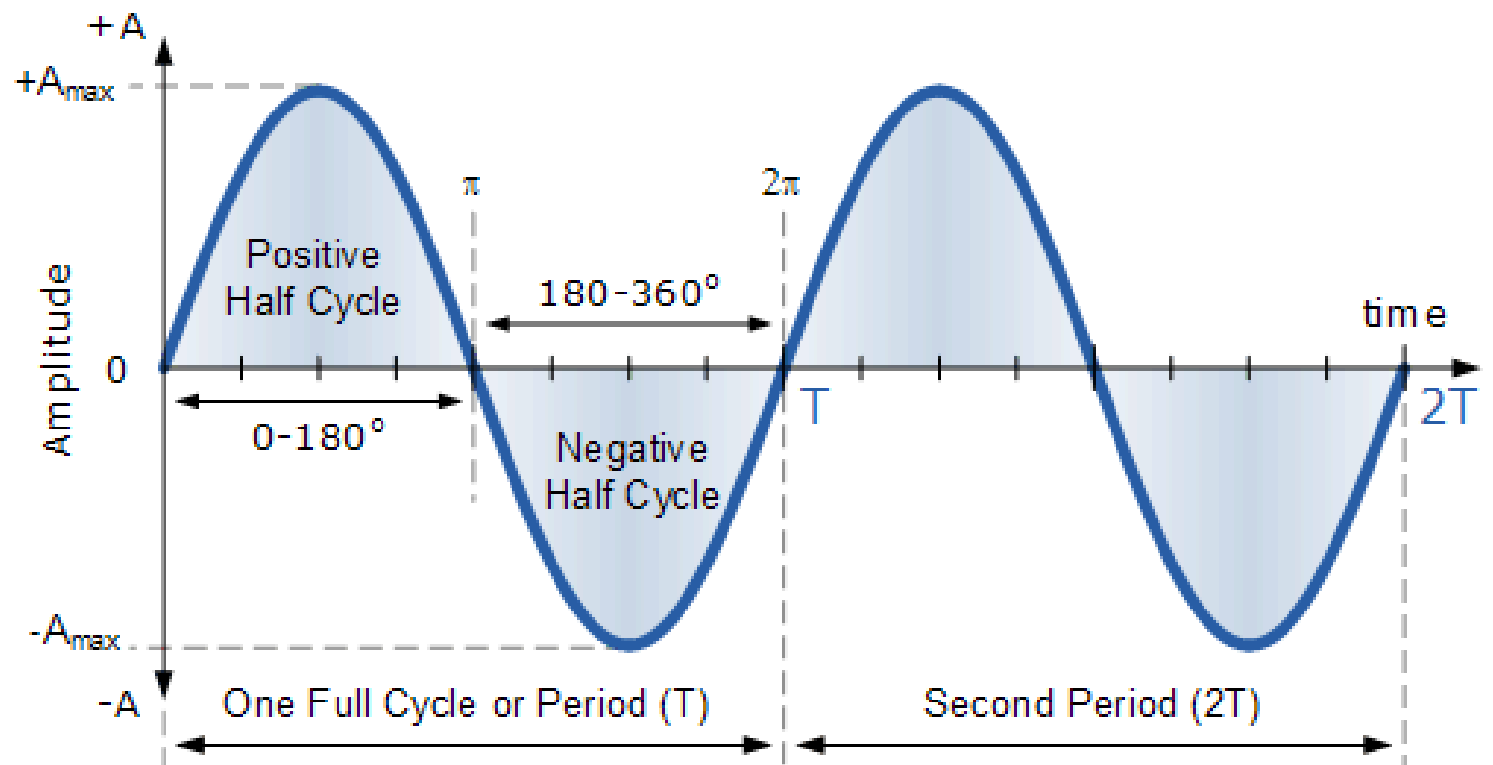
Signal Modulation

- Modulation is the addition of information to an electronic or optical carrier signal.
- It is the process of varying one or more properties of a periodic waveform, the carrier signal, with a modulating signal that typically contains information that is to be transmitted.



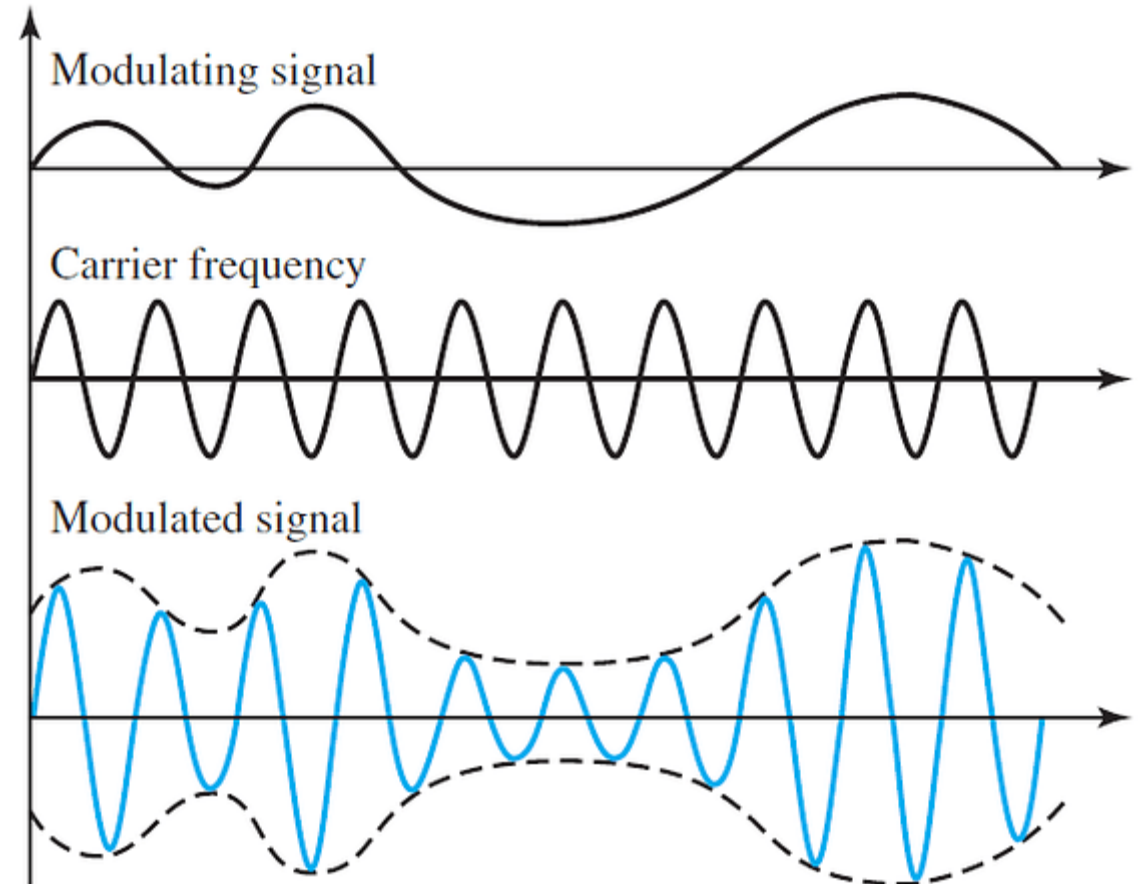
Periodic Wave Properties

- Characteristics of periodic waves
 - Amplitude
 - Frequency
 - Phase



Modulation vs Demodulation

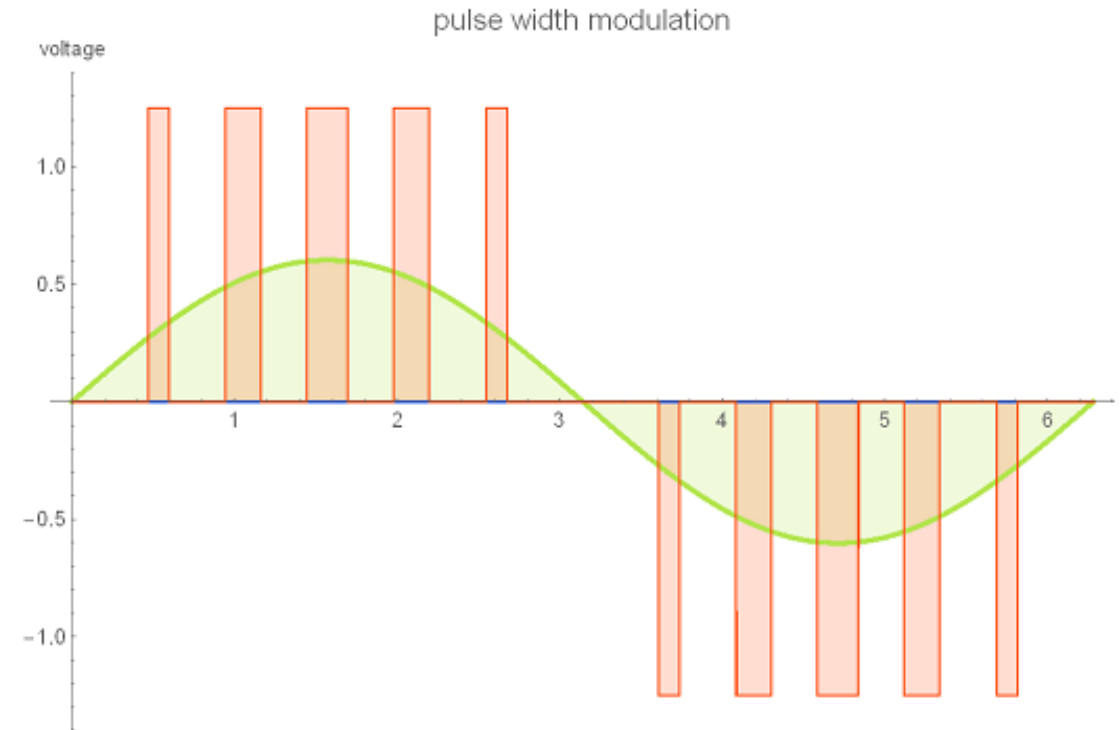
- Process of imposing an input signal on a carrier wave is known as modulation.
- Modulation is similar to hiding code in the carrier wave.
- Demodulation is defined as extracting the original information-carrying signal from a modulated carrier wave.



Modulation + Demodulation = Modem

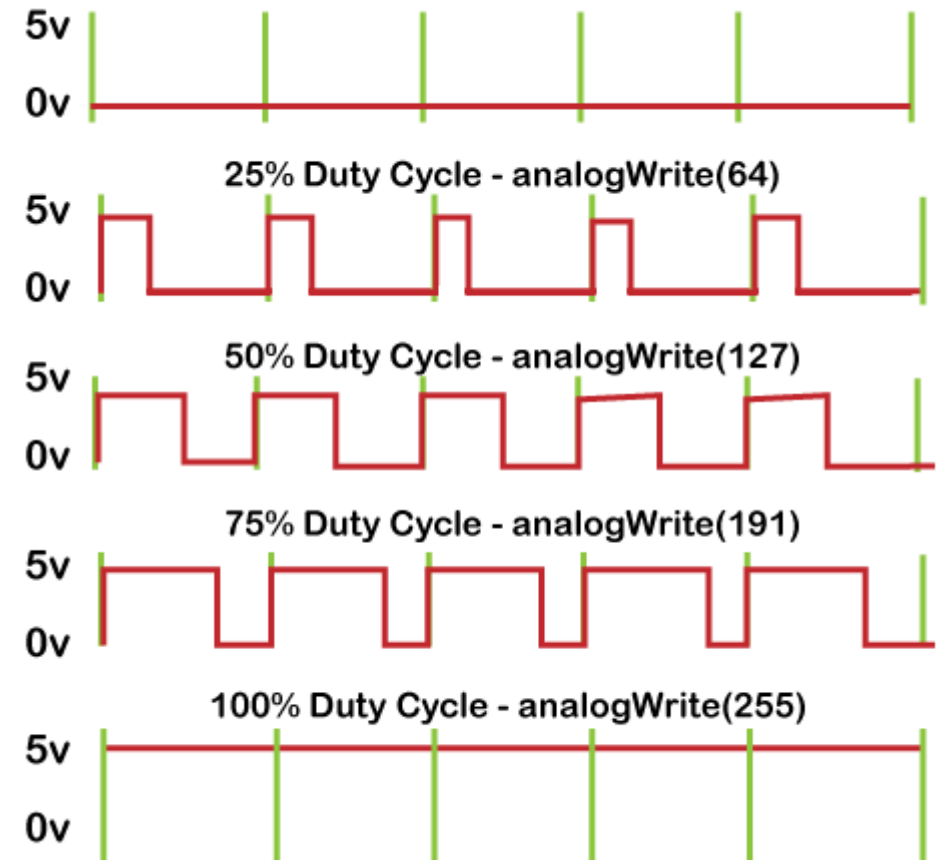
Pulse Width Modulation (PWM)

- PWM is a process to control average power supplied by the electrical signal.
- It is a method in which digital outputs are used to control analogue devices.
 - Speed of motors
 - Smart lightening systems
 - Temperature of the heating elements



Duty Cycle

- The total power applied to the load is directly proportional to the time in which the switch remains turned ON.
- The percentage of the digital signal in which it is turned ON is called the duty cycle.



Software vs Hardware PWM

- HW: With HW PWM, registers are set to specific settings and the HW does the timing. maximum frequency of 19.2 MHz
- SW: With SW, codes set up a pin to act as a PWM pin. The software PWM is available on all 26 of RPi's GPIO and can generate PWM frequencies from 1 Hz up to few kHz.

3.3V	1	2	5V
GPIO2 (SDA1)	3	4	5V
GPIO3 (SCL1)	5	6	GND
GPIO4 (GPIO_GCLK)	7	8	GPIO14 (UART_TXD0)
GND	9	10	GPIO15 (UART_RXD0)
GPIO17 (GPIO_GEN0)	11	12	GPIO18 (GPIO_GEN1) PWM0
GPIO27 (GPIO_GEN2)	13	14	GND
GPIO22 (GPIO_GEN3)	15	16	GPIO23 (GPIO_GEN4)
3.3V	17	18	GPIO24 (GPIO_GEN5)
GPIO10 (SPI0_MOSI)	19	20	GND
GPIO9 (SPI0_MISO)	21	22	GPIO25 (GPIO_GEN6)
GPIO11 (SPI0_CLK)	23	24	GPIO8 (SPI_CE0_N)
GND	25	26	GPIO7 (SPI_CE1_N)
ID_SD (I2C EEPROM)	27	28	ID_SC (I2C EEPROM)
GPIO5	29	30	GND
GPIO6	31	32	GPIO12 PWM0
PWM1 GPIO13	33	34	GND
PWM1 GPIO19	35	36	GPIO16
GPIO26	37	38	GPIO20
GND	39	40	GPIO21

PWM in Python

- For Hardware PWW, Python uses *pigpio* library that gives advanced access to the pins. (*import pigpio*)

- For Software:

Import Rpi.GPIO as GPIO

pulse=GPIO.PWM(output channel , frequency of PWM signal)

pulse.start(DUTYCYCLE)

pulse.ChangeDutyCycle(∞)

LED with PWM

```
import RPi.GPIO as GPIO
```

```
import time                                #calling time to provide delays in program
```

```
GPIO.setwarnings(False)                   #do not show any warnings
```

```
GPIO.setmode(GPIO.BCM) #programming the GPIO by BCM pin  
numbers
```

```
GPIO.setup(12, GPIO.OUT) # initialize GPIO12 as an output
```

```
pulse = GPIO.PWM(12,100)                  #GPIO12 as PWM output, with 100Hz  
frequency
```

```
pulse.start(0)                            #generate PWM signal with 0% duty cycle
```

LED with PWM (Cont.)

<i>while 1:</i>	#execute loop forever
<i>for x in range (75):</i>	#execute loop for 75 times, x being incremented from 0 to 74.
<i> pulse.ChangeDutyCycle(x)</i>	#change duty cycle for varying the brightness of LED.
<i> time.sleep(0.1)</i>	#sleep for 100m second
<i>for x in range (75):</i>	#execute loop for 75 times, x being incremented from 0 to 74.
<i> pulse.ChangeDutyCycle(75-x)</i>	#change duty cycle for changing the brightness of LED.
<i> time.sleep(0.1)</i>	#sleep for 100m second